

An Energy-efficient, Reconfigurable and Dual-mode AI Processor Based on a Two-Stage Hybrid Neural Network for Wearable Lung Sound Monitoring

Keyi Yang, Graduate Student Member, IEEE, Zixuan Shen, Graduate Student Member, IEEE, Zhirui Huang, Bingqiang Liu, Member, IEEE, Heng Ping, Ziyuan Wen, Graduate Student Member, IEEE, Jiahao Liu, Member, IEEE, Jun Zhou, Senior Member, IEEE, Kian Ann Ng, Senior Member, IEEE, and Chao Wang, Senior Member, IEEE

Abstract—Lung diseases significantly impact human health, and using Deep Neural Networks (DNNs) to diagnose related conditions through lung sounds is an emerging trend. Many existing methods focused on direct fine-grained multi-category classification, leading to redundant computation and low energy efficiency in most subject cases with normal lung sound events. Additionally, previous lung sound processors either directly use generic deep-learning Intellectual Property (IP) cores or lack data reuse, resulting in high hardware overhead or low energy efficiency, respectively. This paper proposes an energy-efficient lung sound processor with a two-stage coarse-to-fine DNN model, effectively filtering normal lung sound events before fine-grained classification to reduce unnecessary energy consumption. Three key technologies are as follows: 1) an operator-disparity-driven reconfigurable computing architecture with a dual-mode systolic data flow to accelerate various DNN operators, reducing hardware overhead and improving energy efficiency; 2) a multi-branch hybrid-driven memory architecture to maximize the high energy efficiency inherent in the two-stage hybrid model; and 3) a large-stride layer optimization method to improve hardware utilization and reduce latency. The design is validated on a 16-nm FPGA platform, consuming 0.39 mJ/frame for normal lung sounds and 2.06 mJ/frame for abnormal lung sounds, improving the energy efficiency by 5.28× in most cases with normal events. A 55-nm test chip is then fabricated, which is the first lung-sound AI processor ASIC chip as far as we know from the literature, achieving a high energy efficiency of 2.76 pJ/op at 1.0 V.

Index Terms—Lung sound classification, deep neural network (DNN), AI accelerator, energy efficiency, reconfigurable architecture

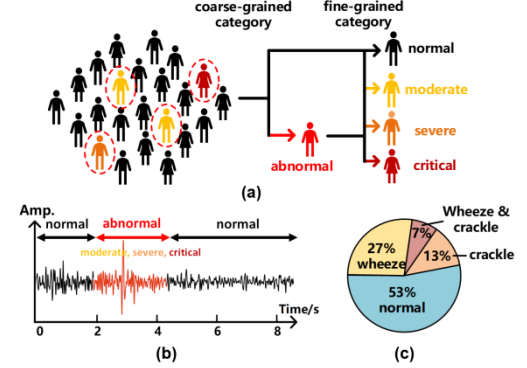


Fig. 1. (a) In general, most people’s lung sounds are normal, while only a small portion are abnormal; (b) Waveform of a lung sound record from ICBHI 2017 Challenge Database [13] including normal and abnormal cycles; (c) Labels distribution of 6898 lung sound cycles in [13]

I. INTRODUCTION

LUNG diseases are one of the leading causes of death around the world. According to the World Health Organization (WHO), lung diseases cause over 3 million deaths each year globally [1]. Early detection and intervention are necessary to reduce the death rate attributed to lung diseases. Auscultation is considered a valuable method for early diagnosis of lung diseases due to its low cost and non-invasive nature [2]. However, conventional auscultation relies on well-trained physicians, leading to the incapability of long-term monitoring and high labor intensity [3].

Lung sound classification using Deep Neural Networks (DNNs) has garnered significant attention due to its excellent performance [4]. Researchers employ Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and hybrid architectures, in conjunction with effective techniques like fine-tuning and data augmentation to improve detection accuracy [5]–[12]. However, there is a challenging trade-off between achieving low-power operation and classification granularity of lung sounds. When performing pre-diagnosis for a large population, it is known that people with abnormal lung sounds are a rarity, as shown in Fig. 1 (a). For patients requiring long-term monitoring, normal and abnormal lung sounds typically occur together in a single recording, as shown in Figs. 1 (b) and (c) [13]. Most lung sounds are normal and only require a simple coarse-grained classification for normal and abnormal detection. In contrast, only a short epoch with

Received XX; revised Apr XX; accepted XX. This article was approved by XX. This work was supported by the National Natural Science Foundation of China under Grants 62274072 and 61974053. (Keyi Yang and Zixuan Shen contributed equally to this work.) (Corresponding author: Chao Wang.)

Keyi Yang, Zixuan Shen, Bingqiang Liu and Chao Wang are with the School of Optical and Electronic Information, Huazhong University of Science and Technology, Wuhan 430074, China. (e-mail: chao.wang.1978@hotmail.com).

Zhirui Huang is with Frontier Institute of Chip and System, Fudan University, Shanghai 200433, China.

Heng Ping is with the Department of Electrical and Computer Engineering, University of Southern California, Los Angeles, CA 90089, USA.

Ziyuan Wen is with the Department of Electrical and Computer Engineering, Rice University, Houston, TX 77005, USA.

Jiahao Liu and Jun Zhou are with the University of Electronic Science and Technology of China, Chengdu 611731, China.

Kian Ann Ng is with Singapore Institute of Technology, Engineering Cluster, 1 Punggol Coast Road, Singapore 828608, Singapore.

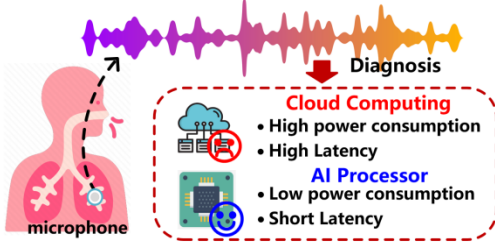


Fig. 2. Edge computing significantly reduces power consumption and latency of lung sound classification

abnormal sounds would require fine-grained multi-category classification, which needs higher computational complexity and power consumption. Although signal processing methods including classical machine learning can be used for abnormal-versus-normal wake-up detection, the detection accuracy is insufficient for reliable screening as studied in [3], [7]. Based on these observations, the Two-Stage Hybrid Neural Network (TS-HNN) model proposed in our previous work [14] is adopted in this study. This model reduces computational workload by filtering dominant normal-sound cases while preserving fine-grained classification capability.

As shown in Fig. 2, many existing works directly transmit recorded lung sound to the cloud for computing [15], [16], which results in substantial on-device power consumption and long round-trip latency. Edge computing utilizing an embedded general-purpose processor, partially addresses these concerns by eliminating the need for raw data transmission, but still suffers from high power and long latency due to the inherent sequential code execution characteristics of general-purpose processor [17]. Therefore, it is necessary to incorporate a custom Artificial Intelligence (AI) accelerator at the edge to reduce power and latency significantly.

Existing on-device AI accelerator solutions can be broadly classified into digital and analog approaches. Although analog implementations can achieve ultra-low power, the sensitivity to PVT variations limits robustness and reliability as required by wearable biomedical monitoring, thereby motivating the adoption of digital AI accelerators [18], [20], [21]. In [18], the researchers have implemented a lung sound classification algorithm on an FPGA by directly adopting Matlab's deep-learning Intellectual Property (IP) core [19], with three independent processing modules for different operators, resulting in low reconfigurability and redundant hardware overhead. Limited by the general IP core, the design in [18] lacks data flow optimization, resulting in limited performance improvement. Furthermore, this design implements two independent accelerators for 3-category and 8-category classification, respectively, resulting in redundant computation and power consumption for the coarse-grained classification. In [20], an FPGA-based accelerator has been customized for classifying lung sounds, which does not utilize data reuse, thereby resulting in substantial data movement and high power consumption. Additionally, it directly supports 5-category classification, resulting in redundant computation and power consumption. To the authors' knowledge, these are only two lung sound AI hardware accelerator designs reported in the

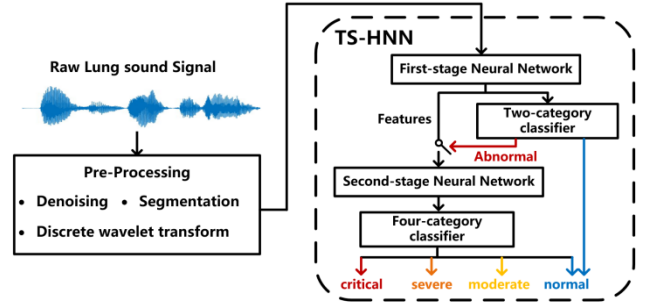


Fig. 3. Flow chart of the Two-Stage Hybrid Neural Network (TS-HNN)

literature. Besides, there is only a cough sound AI hardware accelerator without data reuse techniques reported in [21], which also suffers from low energy efficiency.

In summary, there are three significant issues in the existing works:

- 1) The design in [18] requires dedicated cores for each DNN operator type (e.g., convolution, fully-connected, shortcut), which cannot efficiently handle variable operators, resulting in substantial hardware overhead.
- 2) Most existing designs cannot optimize the data flow due to the design abstraction of deep-learning IP core [18] or have not adopted any data reuse strategies [20], [21], limiting energy efficiency improvement.
- 3) Most lung sound classification designs directly perform fine-grained multi-category classification [18], [20], resulting in redundant computational energy for most normal lung sounds, leading to low energy efficiency.

Based on our previous work in [22], this paper proposes an energy-efficient and reconfigurable dual-mode AI processor for wearable lung sound monitoring systems. As far as we know from the literature, this processor is the first lung-sound AI processor ASIC chip. Together with the Two-Stage Hybrid Neural Network (TS-HNN) algorithm, this AI processor addresses the above issues with the following features:

- 1) An operator-disparity-driven reconfigurable computing architecture is proposed to accelerate DNN operators of variable data sizes and types in the TS-HNN and reduce hardware overhead, integrated with a dual-mode systolic data flow to utilize different data reuse properties and improve energy efficiency.
- 2) A multi-branch hybrid-driven memory architecture is proposed to reuse the feature extracted from the first stage, effectively maximizing the high energy efficiency inherent in the TS-HNN model, while supporting the shortcut and Squeeze-and-Excitation (SE) structure in the second stage.
- 3) A large-stride layer optimization method is proposed to design a custom register array for preloading sliding windows of feature maps in the non-unit stride layers, effectively improving hardware utilization and reducing latency.

The rest of this paper is organized as follows: Section II reviews and analyzes the two-stage hybrid neural network algorithm. Section III presents the hardware architecture and processor design. Section IV presents the performance results

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

for the FPGA and ASIC implementations of our proposed design. Section V concludes this paper.

II. DESCRIPTION AND ANALYSIS OF ALGORITHM

A widely-used public ICBHI'17 dataset [13] and a private TongJi dataset [4] are utilized in this study. The ICBHI'17 dataset contains 126 subjects, with a mean age of 43.0 ± 32.2 years and 63% male participants. It comprises 26 healthy subjects and 100 patients diagnosed with several respiratory diseases, such as chronic obstructive pulmonary disease, bronchiectasis, pneumonia, bronchiolitis, and asthma. Recordings in the ICBHI'17 dataset were sampled at 4 kHz, 10 kHz and 44.1 kHz, and were collected in both clinical and non-clinical environments. The TongJi dataset includes 217 subjects, with a mean age of 64.9 ± 14.9 years and 50% male participants. It consists of 45 healthy subjects and 172 COVID-19 pneumonia patients. All recordings in the TongJi dataset were collected at a 4 kHz sampling rate in a clinical setting. Both datasets are split into training and testing sets using an 80/20 inter-patient strategy.

Fig. 3 illustrates the flowchart of the TS-HNN algorithm [14]. At the beginning, all raw data are downsampled to 4 kHz, denoised to reduce background noise, and segmented into 4-s frames with a 3-s overlap, resulting in a frame size of 1×16000 . After that, a Discrete Wavelet Transform (DWT) with the 'db4' wavelet is employed to extract time-frequency features from the raw data and produce input feature maps of size 4×1000 . The pre-processing module extracts acoustic features to be processed by the TS-HNN, which accounts for less than 6% of the total computational cost.

The first-stage neural network then extracts coarse-grained features, which are fed into a two-category classifier. If the two-category classifier detects normal lung sounds, the TS-HNN output would flag a normal detection signal. Then, no additional energy is consumed within the TS-HNN, thereby achieving high energy efficiency in most cases. On the contrary, if an abnormality is detected, the coarse-grained features extracted by the first stage are sent to the second stage for further classification. Thanks to the two-stage hybrid architecture, the TS-HNN reduces computational complexity and energy consumption in most cases involving normal lung sounds compared with a conventional single-stage four-class model as studied in our previous research in [14].

A. Principle of the TS-HNN Algorithm

The TS-HNN model is inspired by Depth Separable Convolutions (DSC), which feature a lightweight network structure by factoring a standard convolution into depthwise and pointwise convolutions [23]. Fig. 4 illustrates the principles of standard, pointwise, and depthwise convolution, along with their corresponding parameter definitions. Fig. 4 (a) illustrates the standard convolution, which operates on an $H \times W_i \times C_i$ input feature map. It applies the weights, $C_o \times H \times K \times C_i$ to produce an output feature map $H \times W_o \times C_o$, while the pointwise convolution is just a standard convolution with $K=1$. In contrast to the standard and pointwise

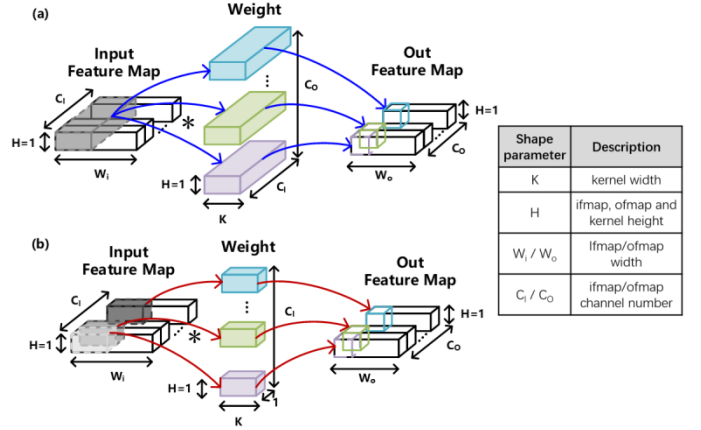


Fig. 4. (a) Standard convolution and Pointwise convolution ($k=1$) and (b) Depthwise convolution

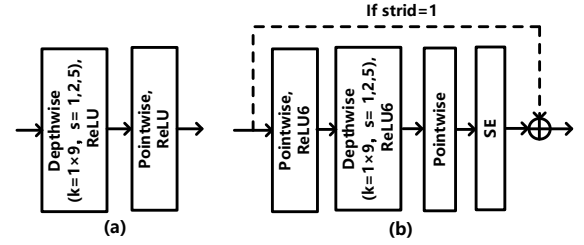


Fig. 5. (a) Basic DSC block used in the first-stage neural network and (b) improved DSC block used in the second-stage neural network

convolutions, the depthwise convolution applies a single kernel of weights to each input channel, as shown in Fig. 4 (b); therefore, $C_o=C_i$ in the depthwise convolution. Notably, since the input data size of the TS-HNN model is $4 \times 1 \times 1000$, the height of all operators shown in Fig. 4 is equal to 1.

Fig. 5 shows the block structure used in the TS-HNN. The first-stage neural network is constructed using basic DSC blocks, as shown in Fig. 5 (a), which can reduce the parameters and computational complexity of coarse-grained classification, thereby enhancing energy efficiency in most cases. As shown in Fig. 5 (b), the second-stage neural network is constructed using improved DSC blocks with an inverted residual structure, which can efficiently alleviate gradient vanishing and increase the model depth for fine-grained classification [24]. Additionally, the second-stage neural network incorporates the Squeeze-and-Excitation (SE) module, which incorporates an attention mechanism to further enhance fine-grained classification accuracy [25].

Table I shows the detailed configuration of the TS-HNN algorithm, while the optimal network configurations for the ICBHI'17 dataset [13] and the TongJi dataset [4] are slightly different from those in [14]. Specifically, basic DSC block3, improved DSC block2/4 in Table I only exist in the network configuration for the TongJi dataset, while the rest are entirely consistent. In the remainder of this paper, the more complex TongJi dataset network configuration is used as an example for algorithm analysis, circuit design, and evaluation. A convolution layer with a large stride of 10 is used in the two-category classifier to achieve effective downsampling and computation reduction [26]. Although a large stride decreases

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

TABLE I
CONFIGURATION OF THE TS-HNN ALGORITHM

	Layer	Kernel	Stride	Input Size	Shortcut	SE
First-stage Neural Network	Conv	4*9	1	4*1000	-	-
	Basic DSC block1	1*9	1	24*1000	-	-
	Basic DSC block2	1*9	2	24*1000	-	-
	'Basic DSC block3	1*9	1	48*500	-	-
	Basic DSC block4	1*9	5	48*500	-	-
Two-category Classifier	Conv	192*11	10	192*100	-	-
	AvgPool	-	-	384*10	-	-
	FC	-	-	384*1	-	-
Second-stage Neural Network	Improved DSC block1	1*9	2	192*100	×	√
	'Improved DSC block2	1*9	1	96*50	√	√
	Improved DSC block3	1*9	5	96*50	×	√
	'Improved DSC block4	1*9	1	192*10	√	√
	Improved DSC block5	1*9	1	192*10	√	√
	Improved DSC block6	1*9	2	288*10	×	√
Four-category Classifier	Conv	384*1	1	384*5	-	-
	AvgPool	-	-	768*5	-	-
	FC	-	-	768*1	-	-

¹Only exists in the network configuration for the TongJi dataset [4]

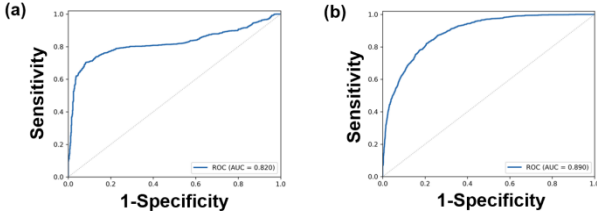


Fig. 6. ROC curves of the first-stage two-category classification based on (a) ICBHI'17 and (b) TongJi dataset

resolution, this effect is minimal for simple, coarse-grained classification tasks and does not impact the four-category classification accuracy due to its branch architecture.

Fig. 6 shows the ROC curves of the first-stage classification on the ICBHI'17 and TongJi datasets, with sensitivities of 77.02% and 82.68%, and false-negative rates of 22.98% and 17.32%, respectively. For long-term monitoring, occasional missed diagnoses might be acceptable as they do not hinder the tracking of overall disease trends [27]. For population screening, sensitivity can be enhanced by slightly lowering the classification threshold, as detailed in Section IV.B.

Table II presents a comparison with relevant existing algorithm works based on the ICBHI'17 dataset. Compared to the open-source works in [5] and [8], the TS-HNN algorithm reduces respectively the amount of computation by 97× and 160× for normal lung sounds, and by 19× and 31× for abnormal lung sounds, while maintaining a comparable ICBHI score. In contrast, the design in [28] adopts an AudioSet-pretrained CNN14 backbone to enhance feature discriminability and improve the detection of abnormal respiratory sounds. This accuracy-oriented approach achieves state-of-the-art highest performance, but at the cost of substantially high computational complexity, making it more suitable for deployment on clouds or portable e-stethoscope. Although TS-HNN achieves moderate accuracy, it achieves

TABLE II
COMPARED WITH PRIOR WORKS ON THE ICBHI'17 DATASET

	2019 BioCAS [5]	2020 TBioCAS [7]	2021 EMBC [8]	2024 ICASSP [28]	Adopted TS-HNN [14]
Task	Only 4-class	Only 4-class	2-class & 4-class	4-class	2-class & 4-class
Features	STFT+ wavelet	Mel-spectrograms	Mel-spectrograms	Mel-spectrograms	DWT
Backbone	Bilinear dual-branch ResNet	Hybrid CNN-BiLSTM	ResNet-34	AudioSet-pretrained CNN14	MobileNetV1/V2-based two-stage CNN
Dataset split (train/test)	90/10	80/20	80/20	80/20	80/20
Split strategy	No Reported	Inter-patient	Inter-patient	No Reported	Inter-patient
ICBHI score ¹	class4:69.3%	class4:66.4%	class2: 77.0% class4: 68.5%	class4:80.4%	class2: 79.2% class4: 63.2%
No. Weights (MB)	1.5	No Reported	21.4	79.6	0.83 (2-class) 3.49 (4-class)
No. Mul	normal lung sound	1.11×10 ⁹	No Reported	1.82×10 ⁹	3.21×10 ¹⁰
	abnormal lung sound	1.11×10 ⁹	No Reported	1.82×10 ⁹	5.96×10 ⁷
No. Add	normal lung sound	1.11×10 ⁹	No Reported	1.82×10 ⁹	3.21×10 ¹⁰
	abnormal lung sound	1.11×10 ⁹	No Reported	1.82×10 ⁹	5.96×10 ⁷

¹ICBHI score= (sensitivity + specificity) / 2

significantly lower computational cost for abnormal and normal lung sounds. This trade-off makes TS-HNN more suitable for always-on, resource-constrained wearable edge devices, where low power consumption, low computational complexity, and real-time processing are essential. Such devices can be used for preliminary screening in remote areas and long-term monitoring, with suspicious cases referred for further clinical examination. Therefore, TS-HNN is selected for the custom hardware implementation in this study.

B. Algorithm Analysis from Hardware Design Perspective

1) Operator Properties Analysis

- Operator Size Disparity:** Table I shows that the TS-HNN model is mainly composed of convolutional layers. By exploiting the operator similarity of different convolutional layers, i.e., sliding-window-based dot product computation, it is possible to accelerate the computation by reconfiguring and reusing the same computing core. Neural network processors typically utilize a systolic Processing Element Array (PEA) [29] as the core, as the local data movement naturally aligns with the inherent data flow of convolution operations. However, the significant disparity of operator sizes in Table I demands flexibility of the computing core. Thus, it is crucial to design a reconfigurable PEA that supports variations in operator sizes within the TS-HNN.
- Data Reusability:** Although power-consuming memory access significantly increases energy consumption for neural network computation, it can be effectively reduced through data reuse [30]. There are three methods of data reuse: filter reuse, convolution reuse, and feature map reuse [31]. Firstly, filter reuse only occurs when multiple input feature maps are processed simultaneously, which does not occur in the frame-by-frame lung sound classification. Secondly, convolution reuse means that one kernel of weights is reused by different sliding windows of a feature map, which exists in all operators shown in Fig. 4. Lastly, in feature map

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

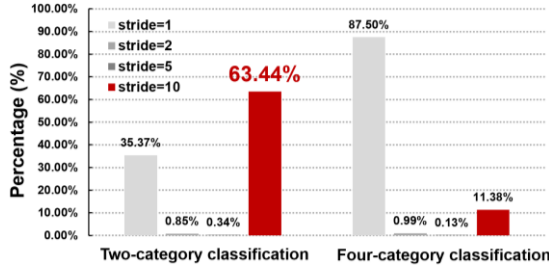


Fig. 7. Breakdown of MAC numbers in the TS-HNN algorithm model across different strides

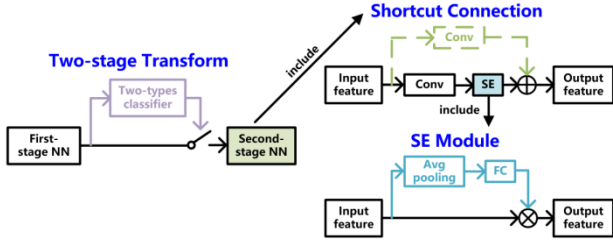


Fig. 8. Multi-branch hybrid structure in the TS-HNN

reuse, one window of a feature map is reused by different kernels, only exhibiting in the standard and pointwise convolutions, but not in the depthwise convolution. Based on this difference, we can categorize the operators into two types by determining whether feature map reuse exists, as shown in Fig. 4 (a) and (b), respectively. If we adopt a fixed data flow, the data reuse properties of different operators will lead to inefficient hardware utilization. To resolve this issue, a multi-mode data flow is designed to improve hardware utilization and reduce computation latency.

- c) **Stride Inconsistency:** The convolution stride also significantly influences the hardware utilization. As the convolution stride is unit in most cases, many existing works are only compatible with unit stride layers [30], [32], [33]. However, as the sliding windows of feature maps required in the non-unit stride layers differ from those in unit stride layers, employing fixed-mode hardware designed for unit stride layers will lower the hardware utilization when computing non-unit stride layers. The larger the stride, the lower the hardware utilization [33]. Fig. 7 shows the breakdown of Multiply-Accumulate (MAC) numbers in the TS-HNN across different strides, where the stride=10 layer has a high proportion that dominates the 2-category classification. Due to the low hardware utilization for a large stride, the hardware performance is seriously degraded for the stride=10 layer, especially in the first-stage 2-category classification. Therefore, developing an optimization method for large-stride layers is necessary to improve hardware utilization and also reduce computation latency.

2) Network Structure Analysis

- a) **Two-stage Mapping Scheme:** For the two-stage network structure in the TS-HNN, there are two mapping schemes: one is mapping each stage to separate computing cores for parallel processing, and the other is mapping both stages to the same computing core for sequential computation. However, the first scheme will lead to high hardware overhead and low hardware utilization, as the second-stage NN remains inactive in most cases with normal lung

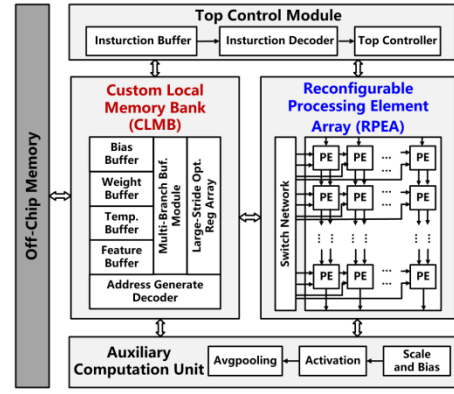


Fig. 9. Hardware architecture of the proposed lung sound AI processor

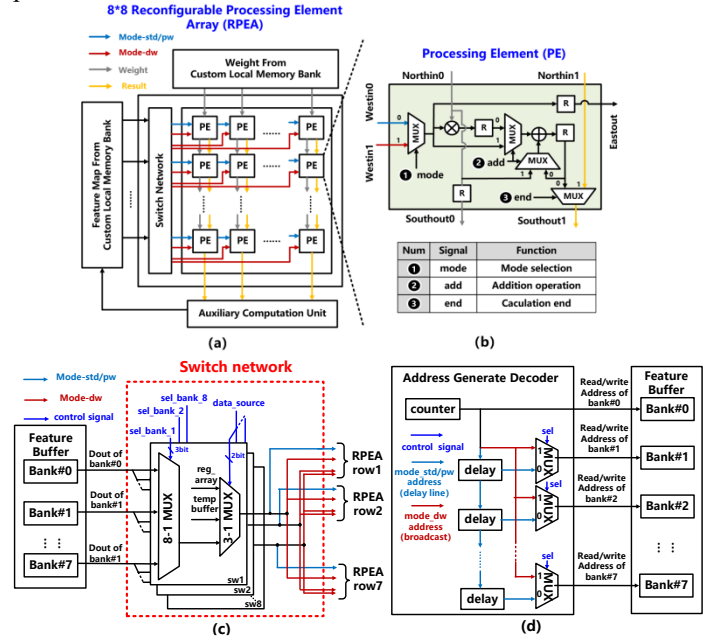


Fig. 10. Hardware architecture of the (a) Reconfigurable Processing Element Array (RPEA), (b) Processing Element (PE), (c) Schematic of switch network and (d) Address generate decoder for feature buffer.

sound. For this reason, the second scheme will be adopted.

- b) **Multi-branch Hybrid Structure:** By reusing the coarse features extracted in the first-stage NN for the second-stage NN, the two-stage hybrid network exhibits inherent high energy efficiency. To maximize this advantage, an additional storage module is required to temporarily buffer the intermediate features. Furthermore, the second-stage NN also contains two hybrid branch structures: shortcut connection and SE module, as shown in Fig. 8. To avoid data conflicts, it is crucial to allocate storage modules reasonably. Therefore, in this study, designing a custom local memory bank with a corresponding data buffering scheme to implement the multi-branch hybrid structure shown in Fig. 8 is necessary.

III. PROPOSED HARDWARE ARCHITECTURE

A. Hardware Architecture of the Proposed AI Processor

Fig. 9 illustrates the hardware architecture of the proposed

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

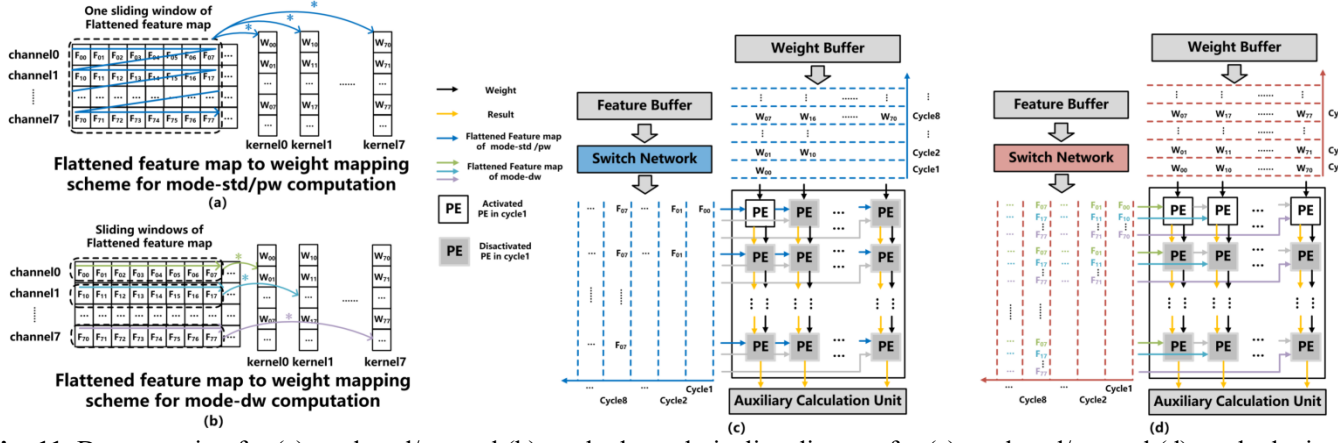


Fig. 11. Data mapping for (a) mode-std/pw and (b) mode-dw and pipeline diagram for (c) mode-std/pw and (d) mode-dw in the dual-mode RPEA

lung sound AI processor, which comprises a top controller, a Custom Local Memory Bank (CLMB), a Reconfigurable Processing Element Array (RPEA), and an auxiliary computation unit. The top controller configures and schedules the CLMB, RPEA, and auxiliary computation unit by a custom instruction set. To implement the TS-HNN with considering a fair comparison with the prior-art designs [18], [20], the quantization-aware training [34] is employed to reduce both feature maps and weights from floating-point precision to 8-bit fixed-point precision. Additionally, all the batch normalization layers are merged into convolution layers [34], further simplifying the hardware design.

B. Reconfigurable and Dual-mode Computing Architecture

1) Reconfigurable Processing Element Array

Fig. 10 illustrates the proposed RPEA, which is organized following the classical systolic-array mapping method [29] and the data-reuse dataflow framework in Eyeriss [30], [31], to efficiently support different operators with significant size disparity, including standard/pointwise/depthwise convolution, fully-connected layer and point-wise addition/multiplication.

The proposed RPEA is designed based on Output Stationary (OS) data flow, which is highly flexible to support operators of different sizes. Specifically, weights and feature maps can be directly flattened and mapped to the time dimension without considering the operator sizes, significantly achieving reconfigurability. In the Weight Stationary (WS) and Input Stationary (IS) data flows, weights or feature maps should be statically mapped to the PEA with a limited array size. Since the weights and feature maps in the TS-HNN have a large variation range (i.e., $1 \times 9 \sim 192 \times 11$ and $1 \times 384 \sim 192 \times 100$, respectively), it is necessary to design a complex operator segmentation and a partial sum scheduling to support the WS and IS data flows, which significantly increases hardware complexity. Therefore, the OS data flow is considered as more balanced and energy efficient in most cases [35].

For the above reasons, this study has utilized an OS-based PEA as the computing core. In the OS data flow, the size K and C_i of a single kernel of weights are mapped to the time dimension of RPEA. The output feature map channel number

(the kernel number) C_o is mapped to the column dimension, while the output feature map width (the sliding window width of the input feature map) W_o is mapped to the row dimension. The fully-connected layer can be mapped to the RPEA as a $K=W_i$ depthwise convolution. For the C_o and W_o parameters of most layers are in multiples of 8, as shown in Table I, the size of RPEA is selected as 8×8 to ensure the C_o and W_o can be divided by the column and row number of RPEA, thereby reducing the idle state of PEs and improving hardware utilization. Notably, the weight buffer and the feature map buffer in the CLMB each comprise 8 banks, aligned with the number of input rows and columns.

Fig. 10 (a) illustrates the hardware architecture of the proposed RPEA. The flattened feature maps and kernel weights are continuously fed into the RPEA, and the MAC operations are performed by the PE units. The partial sums are temporarily stored in the PE units until one convolution operation is completed. Facilitated by a dedicated switch network shown in Fig. 10 (c), the RPEA can be configured into two interconnection modes. Specifically, the switch network comprises eight 8-1 multiplexers and 3-1 multiplexers, which consumes negligible power and hardware overhead (i.e., less than 1% of total power and gate count). The 8-1 multiplexers are used to select one of the 8 banks in the feature buffer, while the 3-1 multiplexers are used to select whether the data comes from the feature buffer, the temp buffer, or the register array. Consequently, a dual-mode dataflow is mapped to the 8×8 RPEA as introduced by Fig. 10.

Fig. 10 (b) shows the schematic of the PE unit. Each PE contains one MAC, four multiplexers for state selection, and four registers for data systolic transmission as well as partial sum temporary storage, with control signals of mode signal, add signal, and end signal. The mode control signal determines the specific input port, which corresponds to one of the two interconnection modes. When a convolution computation is completed, the end signal is flagged, and the result is sent to the auxiliary computation unit without passing through any intermediate registers. Additionally, the PE unit supports element-wise addition and multiplication required for the shortcut connection and the SE module, respectively,

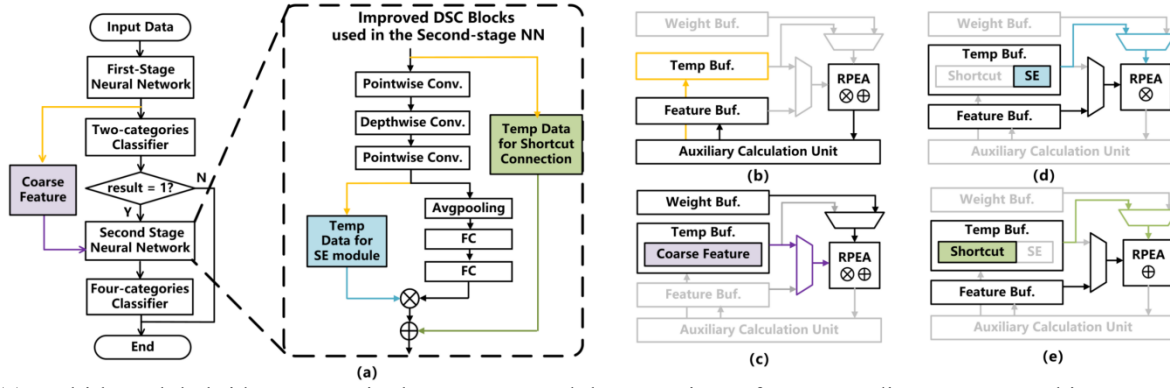


Fig. 12. (a) Multi-branch hybrid structures in the TS-HNN and the operations of corresponding memory architecture when (b) storing temporary data, (c) reading temporary data for the two-stage structure, (d) SE, and (e) shortcut connection.

thereby eliminating extra hardware overhead compared to the prior-art designs [18]. An element-wise multiplication is executed when the end signal is set to 1, and the add signal is cleared to 0. An element-wise addition can be performed when both the end and add signals are set to 1.

2) Dual-mode systolic data flow

Fig. 11 illustrates the proposed dual-mode data flow. By observing the operator-difference data reusability, a dual-mode data flow is designed to reduce data movement and improve energy efficiency for the two operator types, i.e., standard/pointwise convolution and depthwise convolution, as described in Fig. 4. As the algorithm analysis described in Section II.B, convolution reuse exists in both types of operators, while feature map reuse only occurs in the first-type operator. In the RPEA, each kernel of weights is mapped to a specific column, and each sliding window of the feature map is mapped to a specific row. Therefore, in the OS data flow, as defined as the mode-std/pw data flow, the weights are loaded from the top side and then flow through different rows of the RPEA, achieving reuse by the different sliding windows. In contrast, the feature maps are fed from the right side and then flow through the different columns, allowing the reuse by different kernels. However, if mode-std/pw dataflow is still used for the second-type convolution operators, the feature maps cannot be reused by different kernels, resulting in only the first column of PEs being active and low hardware utilization. Thus, the proposed design introduces an additional dataflow, defined as mode-dw mode, which individually flattens each feature map channel and loads them into each column of PEs simultaneously for parallel computation.

As shown in Fig. 11 (a) and (c), the switch network generates the mode-std/pw dataflow following the blue arrows in the flattened feature map. In this case, the RPEA is configured in mode-std/pw, so that the sliding windows of the feature map can be transmitted from left to right, sequentially flowing through each column of PEs. As shown in Fig. 11 (b) and (d), the switch network generates mode-dw dataflow, following the green, light blue and purple arrows in the flattened feature map. In this case, the RPEA is configured in mode-dw for enabling each feature map channel to be sent simultaneously to each column of PEs. Specifically, to support the dual-mode dataflow, the dual-mode stream read/write

access to feature buffer is implemented by using a configurable multiplexer and delay chain to distribute address to different feature-buffer banks, as shown in Fig. 10 (d). In std/pw mode, the address is sequentially propagated through the delay chain to form a staggered access pattern for both read and write operations. In dw mode, it is broadcast simultaneously to enable parallel channel read/write access.

In summary, the proposed dual-mode data flow achieves data reusability. It also prevents the reduction of hardware utilization caused by the different data reuse properties of the two operator types.

C. Multi-branch Hybrid-driven Memory Architecture

Fig. 12 illustrates the multi-branch hybrid structures in the TS-HNN and the corresponding memory architecture, of which the key idea is designing and reusing a temp buffer to store intermediate data to support the two-stage structure and multi-branch hybrid structure as described in Section II.B.

When the branches occur, data that needs temporary storage will be loaded into the temp buffer, as shown by the yellow arrows in Fig. 12 (a) and (b). In the two-stage structure, the coarse features extracted by the first stage are loaded into the temp buffer. When an abnormal lung sound is detected, the coarse features will be sent to the RPEA for the second stage, as indicated by the purple arrows in Fig. 12 (c). To support the shortcut connection and SE module in the second stage, the temporary buffer is reused and divided into two parts: i.e. shortcut buffer and SE buffer, for the shortcut connection and SE module, respectively. Each branch accesses the dedicated buffer, thereby preventing memory access conflicts. At the beginning of the improved DSC block used in the second stage, the input data is both stored in the temporary buffer and sent to the RPEA for convolution operations. Regarding the SE module, the outputs of the pointwise convolution are loaded into the temporary buffer until the average pooling and FC layer operations are completed. Then, as shown in Fig. 12 (a) and (d), the temporary data stored for the SE module is multiplied by the outputs of the FC layer fetched from the feature buffer in RPEA. The results of this multiplication will be reloaded into the feature buffer. Thereafter, as indicated by the green lines in Fig. 12 (a) and (e), the temporarily-stored data for the shortcut connection is added to the multiplication

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

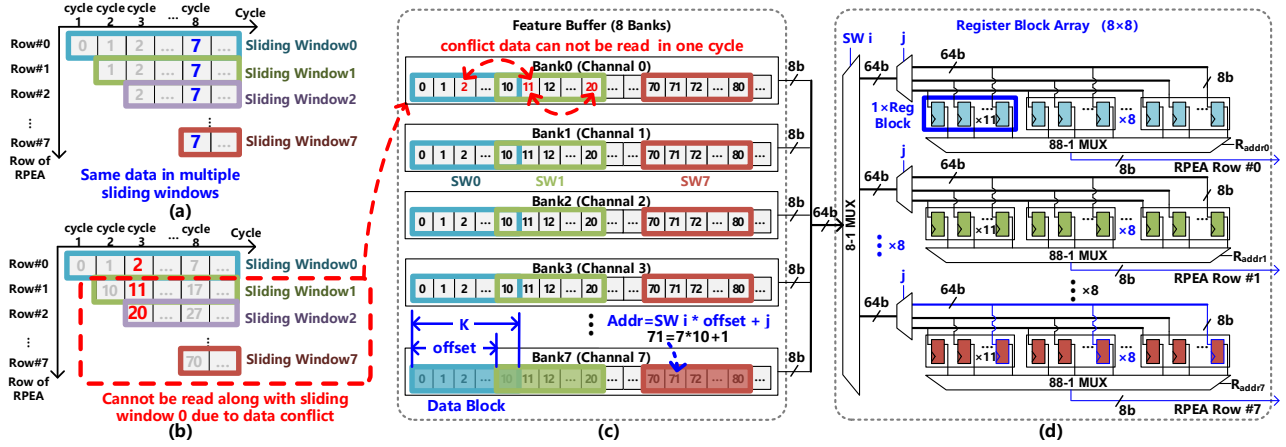


Fig. 13. Sliding windows of feature map to RPEA in the (a) Unit-stride layer, (b) Large-stride layer, (c) Flattened feature map in feature buffer with large stride (e.g., stride=10), and (d) Schematic of register array for large-stride layer optimization.

results of the SE module, thereby achieving the final results.

In this way, the multi-branch hybrid-driven memory architecture enables the second-stage NN to fully leverage the coarse-grained features extracted by the first-stage, achieving high energy efficiency, while supporting the hybrid structure of shortcut connections and SE modules in the second-stage.

D. Large-stride Layer Optimization Method

As shown in Fig. 13 (a), in the unit stride layer, the adjacent sliding windows of a feature map are mapped to the adjacent rows of RPEA. Since the data required for each sliding window in one cycle is the same, one data can be directly broadcast to all rows. However, for the large stride layer (e.g., stride=10), the data required for each sliding window in one cycle is not same at all, as shown in Fig. 13 (b). Therefore, sliding windows 0-7 cannot be read concurrently and sent to the RPEA for fully-parallel processing, i.e., data conflict, resulting in a significant reduction in hardware utilization, as illustrated in Fig. 13 (b) and (c). As the feature map data of each channel is entirely stored in the same bank of the flattened feature map buffer, data conflict occurs when it is required to directly read 2, 11, and 20 from Bank #0 in cycle 3 for maintaining the computation throughput of RPEA.

To address the data conflict, the large-stride layer optimization method is proposed as a general solution, illustrated in Fig. 13 (c). The key idea is to design a dedicated register array that transposes and preloads the data from the feature buffer. Specifically, for the i -th sliding window (SW i), the data address $Addr$ for the same bank can be generalized as:

$$Addr = SW\ i \times offset + j, 0 \leq j < k \quad (1)$$

$$offset = stride$$

where k is the width of sliding window, while j is the index of data within the sliding window. Based on (1), the feature maps in the feature buffer can be represented as a sequence of data blocks, each corresponding to one channel in a sliding window and containing k data elements.

Under this generalized data address, the register array is organized as an 8×8 matrix of register blocks, where each register block consists of k 8-bit registers storing one data block. The raw feature map data stored in the feature buffer is transferred to the register array and reorganized through a

block-level transpose implemented by simple address remapping before being fed into the RPEA. Specifically, the j -th data element of the i -th sliding window across all eight feature-buffer banks, is remapped and stored into the j -th register of each register block in the i -th row of the register array, as illustrated by the blue data bus in Fig. 13 (d).

After the preload process is completed, the reorganized feature map stored in the register array enables eight sliding windows to be simultaneously streamed into the eight rows of the RPEA in a fully-parallel manner, as illustrated in Fig. 13 (b) and (d). To support cases where the number of channels exceeds eight, two register arrays are employed in a ping-pong manner as preloading buffers, allowing one array to be filled while the other is supplying data to the RPEA. This procedure is repeated until all sliding windows have been processed.

In summary, a large-stride layer optimization is proposed to design a custom register array for preloading sliding windows of feature maps in the non-unit stride layers, effectively improving hardware utilization and reducing latency.

IV. FPGA VERIFICATION AND ASIC MEASUREMENT RESULTS

A. Experimental System Setup

As shown in Fig. 14 (a), the quantized input, weights, and biases data are pre-stored in the off-chip memory and loaded into the on-chip memory via a custom memory interface. The bandwidth of the custom memory interface is $64 \times f_{\text{operation}}$ bit/s, where $f_{\text{operation}}$ is the operating frequency of the AI processor. To reduce hardware overhead, weights are not statically stored on-chip but are dynamically moved as illustrated in Fig. 14 (b). During the initialization phase, the input, biases, as well as the weights of layer $n=1$ are preloaded into the on-chip memory. In general, the weights of layer $\#n+1$ are concurrently fetched in parallel with the processing of layer $\#n$, effectively hiding the transmission latency. Only the first-layer weight loading introduces additional latency, accounting for 0.04% and 0.008% of the total latency in the 2-category and 4-category classifications, respectively.

For the evaluated prototype in this study, the bit widths of the parameter registers in the top control module are configured to 4 bits for kernel width, 10 bits for input/output

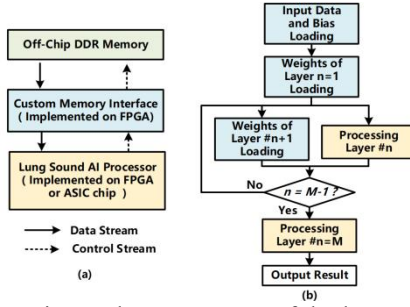


Fig. 14. (a) Experimental system setup of the lung sound AI processor and (b) Processing flowchart of a single frame

feature-map width, 11 bits for input/output channel number, and 4 bits for both stride and padding. The accumulation register in each PE unit is set to 26 bits to satisfy the maximum accumulation length of 2112 (i.e., 192×11) in the TS-HNN configuration. With these settings, all layers listed in Table I can be effectively executed in the evaluated prototype.

B. FPGA Verification Results

The proposed energy-efficient AI processor is first verified on a Xilinx VCU118 FPGA platform to ensure a fair comparison with existing works on FPGA platforms [18], [20], [21] and validated before fabricating the ASIC chip. The on-chip memory size is mainly determined by the layer with the largest parameter size. For FPGA validation, the total memory overhead is 108.5 banks, where each bank is 4.5 KB, i.e., totally 488.25 KB. This includes 64 banks for the feature map buffer, 32 banks for the weight buffer, 4.5 banks for the bias buffer, and 8 banks for the temporary buffer. Operating at 100 MHz and 0.85 V, the FPGA prototype achieves a throughput of 12.8 GOP/s while utilizing 19,151 LUTs, 13,024 FFs, and 73 DSPs.

This study compares the proposed lung sound AI processor with the prior designs in FPGA platforms, including two lung sound classification designs [18], [20] and one related cough sound detection design [21], of which voltage and frequency are normalized to 0.85 V and 100 MHz. Since none of these works exploits sparsity, no sparsity-related normalization is applied. Note that existing lung sound detection tasks can be primarily divided into Respiratory Disease Classification (RDC) [20] and Adventitious Lung Sound Classification (ALSC) [18], focusing on multi-cycle disease recognition and single-cycle anomaly recognition, respectively. As the ALSC task is more widely researched as a standard task in this field [6], both the research in [18] and this study adopt the ALSC.

1) Hardware Overhead

To ensure a fair comparison, the number of LUTs and DSPs is normalized to per GOP/s by dividing the throughput for evaluating the hardware overhead. As Fig. 15 (a) shows, the LUT and DSP cost per GOP/s of the proposed design are 1.50k and 5.70, respectively, which are close to those of the dedicated design in [21], i.e., 0.9k and 2.92, respectively, and also significantly lower than those of the DL IP core-based design in [18], i.e., 22.8k and 35.74. For the BRAM overhead, the proposed design has a model size of 4.07 MB, $2.83 \times$ smaller than the 11.5 MB model in [18], while requiring only 108.5 BRAM banks, $5.37 \times$ fewer than the 583 banks used in [18]. Overall, the hardware overhead of the proposed design is

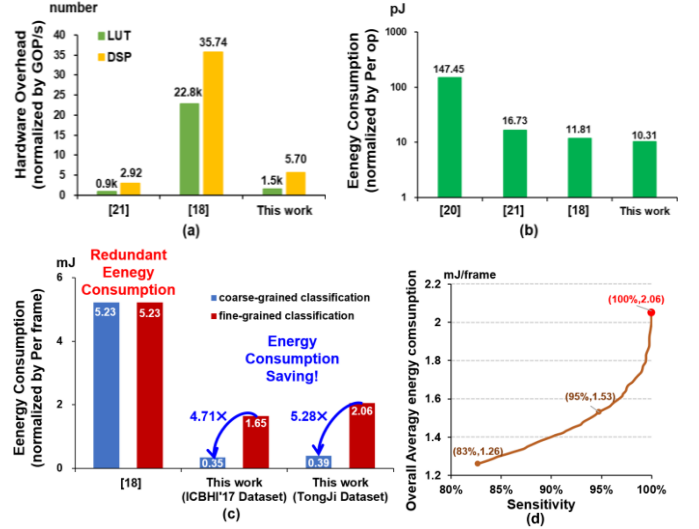


Fig. 15. FPGA verification of the proposed lung sound AI processor: (a) Hardware overhead, (b) Energy consumption per operation, and (c) Energy consumption per frame; (d) Trade-off between the average energy consumption and the sensitivity of the first-stage two-category classification

significantly lower as compared to the prior-art design in [18], which relies on the MATLAB Deep Learning HDL Toolbox to generate multiple processing modules. For accelerating different operators separately, each generated module contains an independent memory bank, a controller, and a non-reconfigurable calculation module, resulting in significantly redundant hardware overhead.

In contrast, our proposed reconfigurable AI design can support multiple operators with various sizes and types within a single computing core by utilizing the dual-mode reconfigurable PE array. Additionally, the memory bank can be shared and reused, thereby significantly reducing the memory hardware overhead. Furthermore, compared to existing two-core approaches, our proposed design reduces hardware overhead by implementing two-stage computation in a single computing core, using the multi-branch hybrid-driven memory architecture.

2) Energy Efficiency

Consistent with most previous lung sound processor designs [18], [20], [21], the FPGA evaluation excludes the energy consumption of off-chip data transfer. Besides, only dynamic power is reported because static power varies significantly across FPGA platforms.

As a cough detection design [21] is included for comparison, the task-independent metric of energy consumption per operation is adopted for fair comparison. The energy consumption per operation of the proposed design is 10.31 pJ/op, which is decreased by $14.30 \times$ and $1.62 \times$, as compared with the two dedicated processors without data-reuse strategy in [20] and [21], respectively. Our design effectively utilizes various data reuse through the proposed dual-mode reconfigurable PE array, thereby significantly reducing data movement and enhancing energy efficiency. Thanks to the algorithm-hardware co-design optimization, our proposed design also achieves 12.7% reduction in energy consumption per operation, as compared to the DL IP core-based design in [18]. As shown in Fig. 15 (b), our design achieves the highest

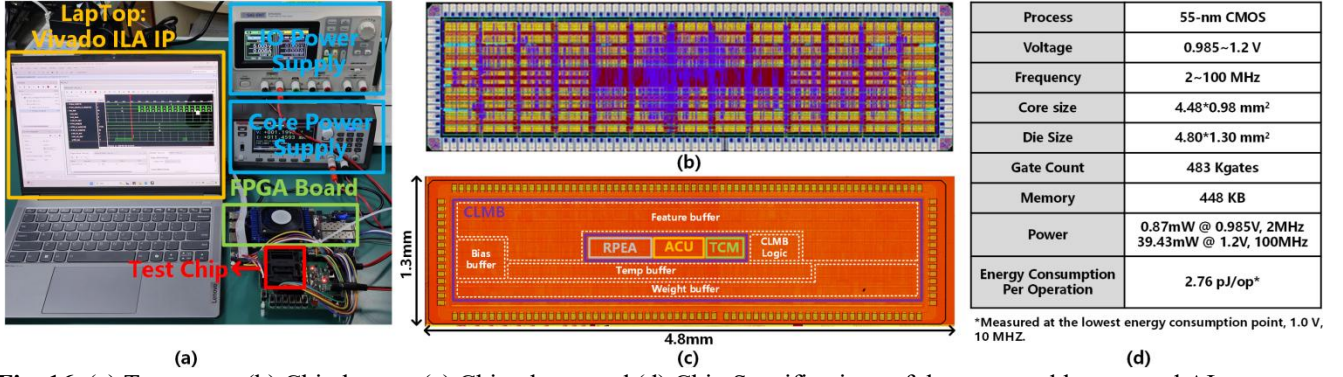


Fig. 16. (a) Test setup, (b) Chip layout, (c) Chip photo, and (d) Chip Specifications of the proposed lung sound AI processor.

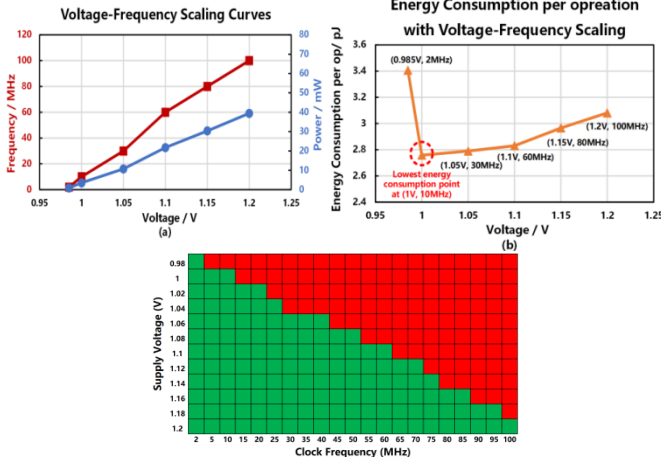


Fig. 17. Measurement results of the test chip: (a) Voltage-frequency scaling curves and (b) Energy consumption with voltage-frequency scaling and (c) Shmoo plot for clock frequency dependence on supply voltage

energy efficiency among the existing designs.

The metric of energy consumption per frame is dependent on the inference task, which can reflect the benefit of the proposed two-stage classification strategy. In our design, compared to fine-grained classification, coarse-grained classification reduces energy consumption per frame by $4.71\times$ (from 1.65 to 0.35 mJ/frame) and $5.28\times$ (from 2.06 to 0.39 mJ/frame) for the ICBHI'17 dataset and the Tongji dataset, respectively. This results in a noticeable reduction in energy consumption in most cases for normal lung sounds. In contrast, the design in [18], which also supports coarse-grained and fine-grained classification, adopts the same network structure with a large model size for multiple-category classifications, thereby resulting in redundant computation and high energy consumption for both normal and abnormal lung sounds. As shown in Fig. 15 (c), our proposed design effectively reduces energy consumption in most cases with normal lung sounds.

Fig. 15 (d) shows the trade-off between the average energy consumption and the sensitivity of the first-stage two-class classifier, using the Tongji test set as an example. The results demonstrate that as sensitivity increases from 83% to 95%, the classification threshold decreases from 0.5 to 0.27. The average energy consumption per frame increases from 1.26 mJ to 1.53 mJ, which remains still lower than the 2.06 mJ for the direct fine-grained classification. Therefore, by adjusting the first-stage 2-category classification threshold, the system can

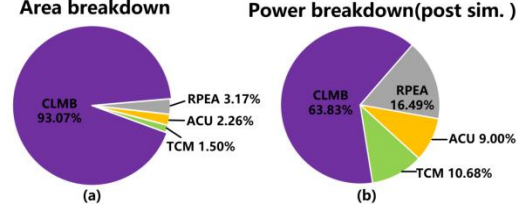


Fig. 18. Breakdown of the test chip: (a) core-level area, (b) core-level power.

be flexibly adapted to actual clinical requirements: long-term monitoring can prioritize low energy consumption to extend battery life, while population screening can be configured with high sensitivity to strictly prevent missed diagnoses.

C. ASIC Chip Measurement Results

The proposed design has been fabricated on a 55-nm CMOS process. The test setup, chip layout, chip photo, and specifications are shown in Fig. 16. An FPGA board is used to transmit input and weight data to the chip, and to capture the output signals by using an Integrated Logic Analyzer (ILA) IP within the Vivado software. Consistent with the FPGA implementation, weights are not entirely stored on-chip but transmitted to the chip layer-by-layer. The die occupies an area of 4.8×1.3 mm² and integrates about 483k gates along with 448-KB SRAM macros generated by the foundry-provided memory compiler. The on-chip memory consists of a 256-KB feature buffer, a 128-KB weight buffer, a 32-KB bias buffer, and a 32-KB temporary buffer. The 32-KB temporary buffer is fully used for the coarse-grained features in the first stage, while the temporary buffer is reused as an 8-KB shortcut buffer and an 8-KB SE buffer to support the multi-branch operations in the second stage. Furthermore, on-chip memory size constrains the complexity of deployable models. Since edge ASIC chips in planar CMOS typically provide only 100–500 KB of on-chip memory [31], high-complexity models usually require frequent off-chip memory accesses. For example, the model in [28] requires 79.6 MB of weights and a maximum intermediate feature map size of 6.6 MB. Compared with the TS-HNN, it requires at least $20\times$ more off-chip memory access, leading to significant data movement and making it unsuitable for low-power wearable edge deployment. A Keithley 2280S-32-6 DC power supply is used to measure the power consumption. The measured power of the test chip

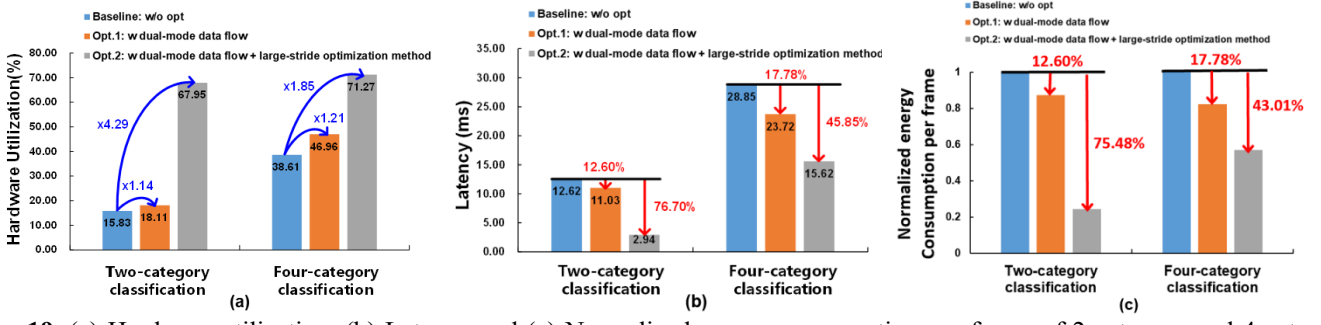


Fig. 19. (a) Hardware utilization, (b) Latency and (c) Normalized energy consumption per frame of 2-category and 4-category classification

core at 1.2 V and 100 MHz is 39.43 mW. The measured lowest energy consumption per operation is 2.76 pJ/op at 1 V and 10 MHz.

Fig. 17 shows the measurement results with voltage-frequency scaling. As shown in Fig. 17 (a), the prototype chip was measured from 0.985 V to 1.2 V, corresponding to a maximum frequency from 2 MHz to 100 MHz with corresponding power consumption from 0.87 mW to 39.43 mW. Note 0.985 V is the minimum voltage required to support a 2 MHz clock frequency that is the minimum frequency to meet the real-time requirement. Fig. 17 (c) shows the Shmoo plot. The supply voltage is swept from 0.98 V to 1.2 V with 0.02 V as the step size. The frequency is increased from 2 MHz to 100 MHz with 5 MHz as the step size. As shown in Fig. 17 (b), the optimal voltage–frequency point for minimum energy consumption per operation is 1.0 V and 10 MHz. This is because the total power is dominated by dynamic power, which is directly proportional to the frequency and the square of the supply voltage. In contrast, processing speed is directly proportional to the frequency. Therefore, energy consumption per operation is typically independent of frequency but decreases with voltage scaling. However, at a low frequency of 2 MHz, the static power becomes non-negligible (i.e., the total power is 0.87 mW, and the static power is 0.35 mW, at 2 MHz and 0.985V). Since the processing time increases with lower frequency, whereas static power is independent of frequency but decreases less with a slight decrement in supply voltage, this results in higher energy consumption per operation at lower clock frequencies.

Fig. 18 (a) and (b) illustrate the core-level area and power breakdown of the chip core, according to the four functional modules presented in Fig. 9. As shown in Fig. 18 (a), the CLMB, which contains SRAM blocks, occupies 93.07% of the total core area, aligning with the chip photo presented in Fig. 16 (c). Fig. 18 (b) presents the post-layout simulated power breakdown obtained from the Synopsys PT-PX tool, where the CLMB and RPEA contribute 63.83% and 16.49% of the total power, respectively. It can be observed that the proposed AI processor solution is a memory-intensive design, which can be optimized in future work by adopting compute-in-memory techniques to reduce on-chip memory access and further improve energy efficiency.

1) Quantitative Evaluation of Proposed Techniques

Fig. 19 (a) and (b) illustrate the hardware utilization and computation latency of 2-category and 4-category classification, respectively. Implementation results across

three configurations are presented, i.e., (1) the baseline without any optimization, (2) optimization 1 using dual-mode data flow, and (3) optimization 2 utilizing both dual-mode data flow and the large-stride layer optimization method.

To fairly evaluate the benefits of both optimization methods, the layer-wise hardware utilization is calculated by

$$HU \text{ of Layer}\#n = \frac{\text{Avg MACs/Cycle}}{\text{Max MACs/Cycle}} = \frac{\text{Layer}\#n \text{ MAC number}}{\text{Layer}\#n \text{ Cycle number} \times 8} \quad (1)$$

where Avg MACs/Cycle is the actual MAC number executed per cycle, and Max MACs/Cycle is the peak MAC number executed by the PE array. The overall utilization of the 2-category and 4-category classification is derived through the weighted sum of each layer's hardware utilization by

$$\text{Total HU} = \sum_{n=1}^M HU \text{ of Layer}\#n \times \frac{\text{Layer}\#n \text{ Cycles}}{\text{Total Cycles}} \quad (2)$$

As presented in Fig. 19, compared to the baseline, the 2-category classification with optimization 1 and 2 has achieved hardware utilization improvements of 1.14 $\times$ and 4.29 $\times$, respectively, with corresponding latency reductions of 12.6% and 76.70%. The large-stride layer optimization method brings more optimization benefits than the dual-mode data flow in the 2-category classification. This is attributed to the stride=10 layer dominating the computation in the 2-category classification, which determines the hardware utilization and computation latency. For the 4-category classification with optimization 1 and 2, hardware utilization has increased by 1.21 $\times$ and 1.85 $\times$, respectively, while computation latency is reduced by 17.78% and 45.85%, compared to the baseline. Compared to the 2-category classification, the benefits of hardware utilization improvement and computation latency reduction for the 4-category classification from the large-stride layer optimization decrease, while the benefits from the dual-mode data flow increase. This is because the proportion of the stride=10 layer in the 4-category classification is lower than that of the 2-category classification. Conversely, the dual-mode data flow is mainly optimizing the depthwise convolution layers. As the 4-category classification has a higher proportion of depthwise convolution layers than the 2-category classification, the benefits from the dual-mode data flow increase in the case of the 4-category classification.

Fig. 19 (c) illustrates the normalized energy consumption per frame of 2-category and 4-category. For optimization 1 that adopts the dual-mode dataflow, the required hardware overhead is limited to around 2.88k gates compared to a fixed dataflow, leading to negligible impact on both chip area and power consumption. For optimization 2 that adopts the large-

TABLE III

COMPARISON WITH THE STATE-OF-THE-ART AI PROCESSOR CHIP DESIGNS FOR COMPLEX ACOUSTIC CLASSIFICATION TASKS

	JSSC'22 [39]	JSSC'20 [37]	CICC'25 [38]	This work	
Technology	65 nm	65 nm	65 nm	55 nm	
Core area	7.8 mm ²	7.74 mm ²	1.7 mm ²	4.39 mm ²	
Application	Video-based Perception	Automatic Speech Recognition	Environmental Sound Classification	Lung Sound Classification	
Two-stage Structure	No	No	Energy-based Threshold Detection	Two-stage hybrid NN Algorithm Model	
Model	Mobilenet	3-layer LSTM	4-layer DSCNN ¹	Mobilenet-based Deep CNN	
				ICBHI'17 dataset: 5-layer (2-class) ¹ 9-layer (4-class) ¹	TongJi dataset: 6-layer (2-class) ¹ 12-layer (4-class) ¹
MAC number of model	Not Reported	3.00 M	16.57 M	ICBHI'17 dataset: 11.41 M (2-class) 59.58 M (4-class)	TongJi dataset: 12.78 M (2-class) 71.27 M (4-class)
Voltage	0.51-1.0 V	0.68 V	0.84 V	0.985-1.2 V	
Data Type	8 IN, 8b W	13b IN, 6b W	8 IN, 8b W	8 IN, 8b W	
Numbers of PE	64	64	Not Reported	64	
On-chip SRAM	196 KB	296.5 KB	24.5 KB	448 KB	
Frequency	25-100 MHz	8 MHz	8.5 MHz	2 -100MHz	
Power ⁴	7.3-99 mW	1.85 mW	0.32 mW ²	0.87-39.43 mW	
Average Energy Consumption Per Operation (Normalized data) ⁸	1.27 pJ/op ³ (3.76 pJ/op) ^{3,7}	0.11 pJ/op ⁴ (0.24 pJ/op) ^{4,7}	2.60 pJ/op ^{2,3,5} (3.68 pJ/op) ^{2,3,7}	ICBHI'17 dataset: 16.67 pJ/op ^{3,3} (2-class) 6.25 pJ/op ^{3,5} (4-class) @ 10 MHz, 1 V	TongJi dataset: 15.23 pJ/op ^{3,3} (2-class) 5.87 pJ/op ^{3,5} (4-class) @ 10 MHz, 1 V
Minimum Energy Consumption Per Operation (Normalized data) ⁸	1.92 pJ/op ³ (5.71 pJ/op) ^{3,7}	Not Reported	Not Reported	2.76 pJ/op ^{3,6} @ 10 MHz, 1 V	

¹ Each DSC block or improved DSC block calculated as one layer; ²Including feature extraction module; ³ Without sparsity; ⁴ Utilizing sparsity; ⁵ Calculated average energy consumption per op = Energy consumption per frame / (MAC numbers of model × 2); ⁶ Calculated minimum energy consumption per op = Power / (number of PEs × 2 × frequency); ⁷ Normalized data = Absolute energy consumption / (Voltage / 1 V)²; ⁸ excluding off-chip data transmission energy

stride optimization method, Post-P&R simulation results indicate that the register array accounts for only 3.19% of the total core area and about 5% of the total power, indicating a limited overhead. In return, the large-stride optimization significantly reduces execution latency by 73.35% and 34.15% and lowers energy consumption per frame by 71.94% and 30.68%, for the 2-class and 4-class classification tasks, respectively, as shown in Fig. 19 (b) and (c). Consequently, despite the small additional hardware cost, optimization 2 achieves significant latency and energy savings, indicating a favorable cost–benefit trade-off.

2) Comparison with State-of-the-Art Acoustics AI Chips

As there are no lung sound dedicated AI chips reported in the literature, Table III presents a comparison with the state-of-the-art acoustic AI chips, including automatic speech recognition [36], [37] and environmental sound classification [38], as well as a recent MobileNet-like CNN accelerator [39]. The results of other works have been normalized to the same operating voltage of 1 V, and the normalized values are presented in parentheses, following the commonly-adopted voltage and frequency scaling law in CMOS circuits [36]. Additionally, all energy-per-operation metrics are based on executed operations [40], except for the design in [37], which only reports energy per effective operation by utilizing sparsity.

In terms of the two-stage structure, the environmental sound AI chips typically use a simple energy-based detection to wake up the main network, thereby consuming negligible

power during silence [36], [37]. In contrast, continuous lung sounds contain richer physiological information, requiring an AI-based coarse-grained classification in the first stage to wake up the more complex classification in the second stage when abnormal event is detected. Additionally, our proposed AI chip exploits a tightly-coupled two-stage NN to fully reuse the first-stage extracted features, thereby eliminating redundant computation, as compared to the conventional acoustic AI chips executing the two classifications separately. Considering the significantly different acoustic tasks, energy consumption per operation is used as the key metric. Our lung sound AI chip is on the same level of energy efficiency as the environmental sound processor in [38]. The speech processor design in [37] achieves much lower energy consumption by aggressively exploiting the network sparsity of 93.75%.

A recent generic CNN accelerator supporting MobileNet-like network models [39] is also included in Table III for a broader comparison. Although the average energy per operation reported in [39] appears extremely low, the results are actually normalized to effective operations [40] by exploiting a sparsity of 90.2%. However, the reported results of our AI chip are based on executed operations without exploiting any sparsity [40]. Under a fully-dense (i.e., 100%) workload, the minimum energy per operation measured in this study is 51.7% lower than the normalized result from the minimum energy per operation without sparsity reported in [39]. These results indicate that the proposed lung sound AI chip achieves competitive energy efficiency among state-of-

the-art acoustic AI processors reported in the literature.

V. CONCLUSION

This paper presents a dual-mode AI processor for wearable lung sound monitoring, based on a Two-Stage Hybrid Neural Network (TS-HNN). The processor's operator-disparity-driven reconfigurable design, dual-mode dataflow, and multi-branch hybrid-driven memory scheme enable efficient coarse-to-fine DNN classification, activating complex analysis only when abnormal lung sounds are detected. The FPGA verification proves significantly higher energy efficiency of the proposed AI processor design against the state-of-the-art solutions. A 55-nm test chip is also fabricated to further validate the proposed lung sound AI processor's energy efficiency for wearable monitoring applications.

REFERENCES

- [1] World Health Organization. (2023). *Chronic Obstructive Pulmonary Disease (COPD)*. [Online]. Available: <https://www.who.int/zh/news-room/fact-sheets/detail/chronic-obstructive-pulmonary-disease-copd>
- [2] S. Leng, R. Tan, K. Chai, C. Wang, D. Ghista and L. Zhong, "The Electronic Stethoscope," *Biomedical engineering online*, vol.14, no.1, Jul. 2015.
- [3] Q. Zhang, J. Zhang, J. Yuan, H. Huang, Y. Zhang, B. Zhang, G. Lv, S. Lin, N. Wang, X. Liu, M. Tang, Y. Wang, H. Ma, L. Liu, S. Yuan, H. Zhou, J. Zhao, Y. Li, Y. Yin, L. Zhao, G. Wang and Y. Lian, "SPRSound: Open-Source SJTU Paediatric Respiratory Sound Database," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 16, no. 5, pp. 867-881, Oct. 2022.
- [4] H. Zhu, J. Lai, B. Liu, Z. Wen, Y. Xiong, H. Li, Y. Zhou, Q. Fu, G. Yu, X. Yan, X. Yang, J. Zhang, C. Wang and H. Zeng, "Automatic Pulmonary Auscultation Grading Diagnosis of Coronavirus Disease 2019 in China with Artificial Intelligence Algorithms: A Cohort Study," *Computer Methods and Programs in Biomedicine*, vol. 213, pp. 106500, 2022.
- [5] Y. Ma, X. Xu, Q. Yu, Y. Zhang, Y. Li, J. Zhao, and G. Wang, "LungBRN: A Smart Digital Stethoscope for Detecting Respiratory Disease Using bi-ResNet Deep Learning Algorithm," in *IEEE Biomedical Circuits and Systems Conference (BioCAS)*, 2019, pp. 1-4.
- [6] T. Nguyen and F. Pernkopf, "Lung Sound Classification Using Co-Tuning and Stochastic Normalization," *IEEE Transactions on Biomedical Engineering*, vol. 69, no. 9, pp. 2872-2882, Sept. 2022.
- [7] J. Acharya and A. Basu, "Deep Neural Network for Respiratory Sound Classification in Wearable Devices Enabled by Patient Specific Model Tuning," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 14, no. 3, pp. 535-544, Jun. 2020.
- [8] S. Gairola, F. Tom, N. Kwatra and M. Jain, "RespireNet: A Deep Neural Network for Accurately Detecting Abnormal Lung Sounds in Limited Data Setting" in *Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC)*, 2021, pp. 527-530.
- [9] J. Xiao, S. Zhu, X. Zhang, Z. Yang, X. Zhong, C. Zou, J. Du, M. Du, C. Ji, H. Wei, Y. Long, X. Ou, X. Chen, Z. Hao, X. Chen, X. Miao and J. Zhou, "A High-Accuracy and Ultra-Energy-Efficient Keyword Spotting Processor with Adaptive Noise Suppression and Multi-User Speaker Verification," *IEEE Journal of Solid-State Circuits*, pp. 1-15, 2025.
- [10] W. Shan, J. Qian, L. Zhu, J. Yang, C. Huang and H. Cai, "AAD-KWS: A Sub- μ W Keyword Spotting Chip with an Acoustic Activity Detector Embedded in MFCC and a Tunable Detection Window in 28-nm CMOS," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 3, pp. 867-876, Mar. 2023.
- [11] J. Liu, J. Fan, Z. Zhong, H. Qiu, J. Xiao, Y. Zhou, Z. Zhu, G. Dai, N. Wang, Q. Liu, Y. Xie, H. Liu, L. Chang and J. Zhou, "An Ultra-Low Power Reconfigurable Biomedical AI Processor with Adaptive Learning for Versatile Wearable Intelligent Health Monitoring," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 17, no. 5, pp. 952-967, Oct. 2023.
- [12] J. Liu, Z. Xie, X. Liu, X. Wang, J. Xiao, C. Guo, J. Fan, Q. Liu, Z. Zhu, S. Li, Z. Zhang, S. Yang, W. Shan, S. Lin, L. Zhou, L. Chang, S. Liu and J. Zhou, "A High-Accuracy and Ultra-Energy-Efficient Cardiac Arrhythmia Classification Processor for Wearable Intelligent ECG Monitoring," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 10, pp. 3762-3778, Oct. 2025.
- [13] B. M. Rocha, D. Filos, L. Mendes, G. Serbes, S. Ulukaya, Y. P. Kahya, N. Jakovljevic, T. L. Turukalo, I. M. Vogiatzis, E. Perantoni, E. Kaimakamis, P. Natsiavas, A. Oliveira, C. Jácóme, A. Marques, N. Maglaveras, R. P. Paiva, I. Chouvarda and P. Carvalho, "An open access database for the evaluation of respiratory sound classification algorithms," *Physiological measurement*, vol.40, no. 3, pp. 035001, Mar. 2019.
- [14] B. Liu, Z. Wen, H. Zhu, J. Lai, J. Wu, H. Ping, W. Liu, G. Yu, J. Zhang, Z. Liu, H. Zeng and C. Wang, "Energy-Efficient Intelligent Pulmonary Auscultation for Post COVID-19 Era Wearable Monitoring Enabled by Two-Stage Hybrid Neural Network," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2022, pp. 2220-2224.
- [15] Y. Ma, X. Xu and Y. Li, "LungRN+ NL: An Improved Adventitious Lung Sound Classification Using Non-Local Block ResNet Neural Network with Mixup Data Augmentation," *Interspeech*, pp. 2902-2906, 2020.
- [16] J. Li, J. Yuan, H. Wang, S. Liu, Q. Guo, Y. Ma, Y. Li, L. Zhao and G. Wang, "LungAttn: Advanced Lung Sound Classification using Attention Mechanism with Dual TQWT and Triple STFT Spectrogram," *Physiological Measurement*, vol. 42, no. 10, pp. 29-42, Oct. 2021.
- [17] J. Liu, X. Liu, X. Wang, Z. Xie, C. Guo, Z. Zhong, J. Fan, H. Qiu, Y. Xu, H. Qin, Y. Long, Y. Zhou, Z. Shen, L. Zhou, L. Chang, S. Liu, S. Lin, C. Wang and J. Zhou, "A High Accuracy and Ultra-Energy-Efficient Zero-Shot-Retraining Seizure Detection Processor," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 11, pp. 3549-3565, Nov. 2024.
- [18] J. Hu, C. S. Leow, S. Tao, W. L. Goh and Y. Gao, "Supervised Contrastive Learning Framework and Hardware Implementation of Learned ResNet for Real-time Respiratory Sound Classification," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 19, no. 1, pp. 185-195, Feb. 2025.
- [19] Mathworks. (2024). *Deep Learning Processor IP Core Architecture*. [Online]. Available: <https://ww2.mathworks.cn/help/deep-learning-hdl/ug/deep-learning-processor-architecture.html>
- [20] A. N. Mazumder, H. Ren, H. Rashid, M. Hosseini, V. Chandraredy, H. Homayoun and T. Mohsenin, "Automatic Detection of Respiratory Symptoms Using a Low-Power Multi-Input CNN Processor," *IEEE Design & Test*, vol. 39, no. 3, pp. 82-90, Jun. 2022.
- [21] P. Peng, K. Jiang, M. You, J. Xie, H. Zhou, W. Xu, J. Lu, X. Li and Y. Xu, "Design of an Efficient CNN-Based Cough Detection System on Lightweight FPGA," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 17, no. 1, pp. 116-128, Feb. 2023.
- [22] K. Yang, B. Liu, Z. Huang, H. Ping, Z. Wen, Z. Shen, C. Huang, C. Wang, "An Energy-efficient and Dual-mode AI Accelerator for Wearable Lung Sound Monitoring," in *IEEE Biomedical Circuits and Systems Conference (BioCAS)*, 2024, pp. 1-5.
- [23] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto and H. Adam, "Mobilenets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017, *arXiv:1704.04861*.
- [24] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L. C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510-4520.
- [25] A. G. Howard, M. Sandler, G. Chu, L. C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le and H. Adam, "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314-1324.
- [26] R. Rachid, O. Teboul, D. Grangier and N. Zeghidour, "Learning Strides in Convolutional Neural Networks," 2022, *arXiv:2202.01653*.
- [27] J. Leenen, C. Leerentveld, J. Dijk, H. Westreenen, L. Schoonhoven, G. Patijn, "Current evidence for continuous vital signs monitoring by wearable wireless devices in hospitalized adults: Systematic review," *Journal of medical Internet research*, vol. 22, no. 6, pp. e18636, Jun. 2020.
- [28] A. Chang, J. Tzeng, H. Chen, C. Sung, C. Huang, E. Huang, C. Lee, "GaP-Aug: Gamma Patch-Wise Correction Augmentation Method for Respiratory Sound Classification," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 551-555.

- [29] K. K. Parhi, VLSI digital signal processing systems: design and implementation. John Wiley & Sons, 2007.
- [30] Y. H. Chen, T. Krishna, J. S. Emer and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127-138, Jan. 2017.
- [31] V. Sze, Y. H. Chen, T. J. Yang and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295-2329, Dec. 2017.
- [32] L. Liu and S. Brown, "Leveraging Fine-grained Structured Sparsity for CNN Inference on Systolic Array Architectures," in *International Conference on Field-Programmable Logic and Applications (FPL)*, 2021, pp. 301-305.
- [33] C. H. Duk, "RiSA: A Reinforced Systolic Array for Depthwise Convolutions and Embedded Tensor Reshaping," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 20, pp. 1-20, 2021.
- [34] N. Markus, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. Baalen and T. Blankevoort, "A White Paper on Neural Network Quantization," 2021, *arXiv:2106.08295*.
- [35] S. Choi, J. Sim, M. Kang, Y. Choi, H. Kim and L. S. Kim, "An Energy-Efficient Deep Convolutional Neural Network Training Accelerator for In Situ Personalization on Smart Devices," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 10, pp. 2691-2702, Oct. 2020.
- [36] S. Zheng, P. Ouyang, D. Song, X. Li, L. Liu, S. Wei and S. Yin, "An Ultra-Low Power Binarized Convolutional Neural Network-Based Speech Recognition Processor with On-Chip Self-Learning," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 66, no. 12, pp. 4648-4661, Dec. 2019.
- [37] D. Kadetotad, S. Yin, V. Berisha, C. Chakrabarti and J. S. Seo, "An 8.93 TOPS/W LSTM Recurrent Neural Network Accelerator Featuring Hierarchical Coarse-Grain Sparsity for On-Device Speech Recognition," *IEEE Journal of Solid-State Circuits*, vol. 55, no. 7, pp. 1877-1887, July 2020.
- [38] L. Peng, X. Jiao, Z. Chen, J. Yang, R. Hong, L. Yan, Y. Long, X. Chen, X. Miao, Z. Wang, Z. Wang, L. Zhou, L. Chang, S. Liu, T. H. Kim and J. Zhou, "A High Accuracy and Ultra-Low Energy Environmental Sound Recognition Processor with Progressive Spectrogram Processing and Adaptive Weight Clustering Based Online Learning," in *IEEE Custom Integrated Circuits Conference (CICC)*, 2025, pp. 1-3.
- [39] Y. Yang, Y. Liu, Z. Yuan, W. Sun, R. Liu, J. Wang, J. Yue, X. Feng, Z. Yuan, X. Li and H. Yang, "A 65-nm Energy-Efficient Interframe Data Reuse Neural Network Accelerator for Video Applications," in *IEEE Journal of Solid-State Circuits*, vol. 57, no. 8, pp. 2574-2585, Aug. 2022.
- [40] G. Burr, S. Lim, B. Murmann, R. Venkatesan and M. Verhelst, "Fair and Comprehensive Benchmarking of Machine Learning Processing Chips," in *IEEE Design & Test*, vol. 39, no. 3, pp. 18-27, June 2022.